

Email Summarizer Agent

A personal learning project documenting the tools, services, and architecture behind an AI-powered email agent built with Python and the Gemini API.

Project: Email Service | **Stack:** Python, Gmail API, Gemini API | **Purpose:** Learning project (career transition to AI engineering)

1. Project Overview

This project is a personal email assistant that reads a Gmail inbox and produces AI-generated summaries. It exists in two versions:

- **v1 – Fixed pipeline** (`main.py` + `summarizer.py`): you specify a Gmail search query, the script fetches matching emails and asks the model to summarize them. No decision-making happens on the model's side beyond the summarization itself.
- **v2 – Agent** (`main_agent.py` + `agent.py`): you give the model a plain-English goal (e.g. “catch me up on anything urgent”), and the model decides what Gmail search queries to run, executes them via a tool it calls itself, and can search more than once before producing a final answer.

The difference between the two is the core concept this document explains: moving from code that *tells* an LLM what to do, to code that gives an LLM a *goal* and a *tool*, and lets it decide the steps.

2. Google OAuth — Purpose & Role in This Project

What it is

OAuth 2.0 is an authorization protocol — a standard way for one application (this project) to be granted limited access to a person's account on another service (Gmail), **without ever seeing or storing that person's password**.

Why this project needs it

The agent needs to read real emails from a real Gmail inbox. Google will not allow any script to simply log in and read mail directly — every third-party access request must go through OAuth, so the account owner can explicitly approve (or later revoke) exactly what the app is allowed to do.

How it's used here

- **credentials.json** — downloaded from Google Cloud Console. Identifies this specific app to Google's OAuth servers (its “client ID” and “client secret”).
- **Scope:** `gmail.readonly` — the project deliberately requests read-only access. The agent can never send, delete, or modify email, only read it.
- **First run** — opens a browser window, the user logs into their own Google account and approves the requested scope.
- **token.json** — after approval, an access token (and refresh token) is saved locally, so the browser login step isn't needed again. The token silently refreshes itself on future runs.

File in project: `gmail_auth.py`

3. Google AI Studio — Purpose & Role in This Project

What it is

Google AI Studio (`aistudio.google.com`) is Google's web console for experimenting with and obtaining access to Gemini models. It's the Gemini equivalent of what the OpenAI Platform dashboard is for GPT models.

Why this project needs it

This is where the **Gemini API key** is generated — the credential that authorizes this project's code to send requests to Google's Gemini models and be billed (or, for light personal use, stay within the free tier).

How it's used here

- A free API key is created once at `aistudio.google.com/apikey`.
- The key is stored in a local `.env` file as `GEMINI_API_KEY` — never hardcoded into source files, and never committed to version control.
- The key authenticates every call this project makes to the Gemini API (both the plain summarization calls in `v1`, and the agent's tool-calling calls in `v2`).

File in project: `.env` (created locally, not shared) / referenced in `summarizer.py` and `agent.py`

4. Google GenAI SDK — Purpose & Role in This Project

What it is

`google-genai` is Google's official current Python library for calling Gemini models. It replaced an older, now-legacy package called `google-generativeai` (used briefly early in this project, then migrated away from).

Why this project needs it

It's the code-level bridge between this project's Python and Google's Gemini models running in the cloud — handling the HTTP requests, authentication, and response parsing so the project code can just call Python functions.

Two APIs within this SDK, and why v2 uses the newer one

- `client.models.generate_content()` — the classic, simpler call-and-response API. Used in `v1`'s `summarizer.py`. Works well for single-shot summarization with no back-and-forth.
- `client.interactions.create()` (the “Interactions API”) — a newer, stateful API designed for multi-turn tool-calling conversations. Used in `v2`'s `agent.py`. Google's newer “thinking” models attach an encrypted reasoning token (a *thought signature*) to their responses that must be correctly preserved across turns of a tool-calling conversation — the Interactions API manages this automatically on Google's servers, which is why `v2` was rebuilt around it after the older approach kept failing.

5. Tool / Function Calling — The Core Agent Mechanism

What it is

Function calling (a.k.a. “tool use”) lets an LLM, instead of only returning text, respond with a structured request to run a specific function with specific arguments. The application executes that real function and returns the result to the model, which can reason further or answer.

The tool defined in this project

Tool name: `search_emails`

- **Input:** a Gmail search query string (e.g. `is:unread, newer_than:2d`) and an optional result limit.
- **What it actually does:** calls the same real, read-only Gmail-fetching code used in v1 (`gmail_client.py`) — there is no simulated or fake data. When the model calls this tool, it is querying the user's real inbox.
- **Output:** a formatted block of matching emails (sender, subject, date, and a truncated body) returned as text back to the model.

The agent loop

The loop implemented in `agent.py` follows this pattern:

1. Send the user's goal + tool definition to the model
2. If the model responds with a `function_call` step:
 - > run the real Python function
 - > send the result back, continuing the same conversation
 - > repeat (model may call the tool again with a different query)
3. If the model responds with plain text instead:
 - > that is the final answer, stop the loop
4. Safety cap: stop after 5 rounds regardless, to prevent infinite loops

This loop is the essence of what almost every “AI agent framework” (LangGraph, CrewAI, the various vendor Agent SDKs) does under the hood — they just add convenience, retries, and multi-agent coordination on top of this same request → tool call → tool result → repeat pattern.

6. Summary: Every Tool & Service Used

Tool / Service	Category	Purpose in this project
Google Cloud Console	Setup / Console	Where the Gmail API is enabled and OAuth client credentials (<code>credentials.json</code>) are created.
Google OAuth 2.0	Authentication	Grants this project read-only, revocable access to the user's real Gmail inbox without ever seeing the password.
Gmail API	External API	The actual service queried to list and fetch email messages and their content.
Google AI Studio	Setup / Console	Where the free Gemini API key is generated, authorizing calls to Gemini models.

Tool / Service	Category	Purpose in this project
Gemini API	External API / LLM	The large language model that performs summarization (v1) and tool-based reasoning (v2).
google-genai SDK	Python Library	Official Python client used to call the Gemini API and (in v2) the Interactions API for stateful tool calling.
google-auth-oauthlib	Python Library	Handles the OAuth 2.0 login flow (opening the browser, exchanging the auth code for tokens).
google-api-python-client	Python Library	Generic Google API client used here to build the Gmail service object and call its endpoints.
python-dotenv	Python Library	Loads the GEMINI_API_KEY (and other config) from the local .env file into the running program.
beautifulsoup4	Python Library	Strips HTML markup out of HTML-formatted emails so only clean, readable text reaches the model.
search_emails (custom tool)	Agent Tool	The one tool given to the v2 agent; lets the model decide what Gmail query to run to fulfill the user's goal.

7. Project File Map

File	Version	Responsibility
gmail_auth.py	Shared	Runs the OAuth login flow; saves/refreshes token.json.
gmail_client.py	Shared	Fetches and cleans emails from the Gmail API (strips HTML, quoted replies).
summarizer.py	v1	Sends a fixed batch of emails to Gemini for a one-shot summary.
main.py	v1	CLI entry point; you supply the Gmail query directly (--query).
agent.py	v2	Defines the search_emails tool and runs the model's tool-calling loop via the Interactions API.
main_agent.py	v2	CLI entry point; you supply a goal in plain English, the agent decides the rest.
.env	Shared (local only)	Holds the Gemini API key and model name. Never shared or committed.
credentials.json	Shared (local only)	OAuth client identity downloaded from Google Cloud Console.

File	Version	Responsibility
token.json	Auto-generated	Saved Gmail access/refresh tokens after first login.
requirements.txt	Shared	Pinned Python dependency versions for reproducible installs.

8. Key Concepts Glossary

Term	Meaning
Pipeline	A fixed sequence of steps with no decision-making by the AI (v1's design).
Agent	A system where the LLM decides what actions to take (e.g. which tool to call, with what arguments) to achieve a goal (v2's design).
Tool / Function calling	A structured way for an LLM to request that a real function be executed on its behalf, then receive the result.
Thought signature	An encrypted token some Gemini models attach to their reasoning steps; must be preserved across turns of a multi-step tool-calling conversation.
Stateful conversation	A conversation where the server (not your code) remembers prior turns, referenced via an ID such as <code>previous_interaction_id</code> .
Scope (OAuth)	The specific, limited set of permissions granted to an app — this project only ever requests <code>gmail.readonly</code> .